Project Summary

ZecDev Launchpad is a Linux first toolkit that brings up a Zebra regtest network with a faucet, Unified Address fixtures, and your choice of lightwalletd or Zaino, plus a reusable GitHub Action that runs golden end to end shielded flows on every pull request. It closes a critical gap as zcashd is deprecated by giving every wallet, SDK, and service a fast repeatable path for local development and CI that catches breakage before users do.

Project Description

ZecDev Launchpad is a developer-first toolkit and CI harness that provides a fast, reliable, and unified environment for building on Zebra, the new Zcash node implementation replacing `zcashd`. It allows developers to spin up a complete local devnet, including Zebra (regtest), a faucet, Unified Address (ZIP-316) fixtures, and either lightwalletd or Zaino; all with a single command. The same setup is available as a GitHub Action, letting any repository automatically run end-to-end shielded transaction tests on every pull request.

The goal of ZecDev Launchpad is to make Zcash development faster, more consistent, and less error-prone by standardizing the testing and integration workflow in the post-zcashd era. It eliminates fragmented local setups, ensures backend compatibility between lightwalletd and Zaino, and helps projects detect wallet or transaction regressions before code is merged. By providing this unified developer and CI layer, ZecDev Launchpad strengthens the foundation for future Zcash ecosystem growth and developer onboarding.

Proposed Problem

Proposed Problem

Zcash is in a critical migration window: `zcashd` is being deprecated in 2025 and full-node development is moving to Zebra (`zebrad`). Yet builders lack a standard, maintained, developer-first way to bring up a Zebra + light-client backend stack and run repeatable end-to-end shielded tests locally and in CI. In practice, teams are stuck with fragile one-off scripts, flaky test rigs, and regressions that users discover first which is the exact opposite of what this transition requires.

- Migration pressure is immediate. The deprecation is official; projects are expected to migrate to Zebra. Without a unified DX/CI layer, each repo keeps rebuilding the same plumbing which happens slowly, inconsistently, and with divergent assumptions.
  • Zcash deprecation notice: https://z.cash/support/zcashd-deprecation/
- Tooling exists, but it's scattered; not standardized for DX/CI. Zebra documents how to run lightwalletd and even ships an example `docker-compose.lwd.yml`, but this is not a complete developer workflow: no pre-mined funds, no UA fixtures, no golden flows, no artifacted logs, and no reusable PR-level harness. Teams must stitch pieces together, which leads to drift and brittleness.
  • Zebra + lightwalletd docs: https://zebra.zfnd.org/user/lightwalletd.html
  • Zebra with Docker (compose example): https://zebra.zfnd.org/user/docker.html
  • Example compose file (testing flags noted): https://raw.githubusercontent.com/ZcashFoundation/zebra/main/docker/docker-compose.lwd.yml
- Backends are in flux, projects need parity now. Many stacks still use lightwalletd, while Zaino is an emerging Rust indexer serving light clients directly from Zebra. Builders need a single harness that runs against both backends so they can migrate without losing coverage. Today there is no toggleable devnet or parity test suite for this.
  • Zaino (project docs): https://zingolabs.github.io/zaino/
  • Zaino (GitHub): https://github.com/zingolabs/zingo-proxy
- Unified Addresses (UA) are the default surface and must be tested end-to-end. ZIP-316 defines UAs, and guidance emphasizes UA as the modern address format but most repos lack UA fixtures and golden flows (generate UA → fund → autoshield → shielded send → rescan/sync → verify). This gap hides interoperability bugs until late.
  • ZIP-316 (Unified Addresses): https://zips.z.cash/zip-0316
  • ECC blog explainer: https://electriccoin.co/blog/unified-addresses-in-zcash-explained/
- Cross-platform friction is real; Linux is the practical baseline. Even with Docker Desktop 4.34+ adding host-networking support, macOS/Windows remain nuanced for networked devnets; most CI runs on Linux. Teams need a Linux-first solution that "just works," with Desktop caveats documented and not another round of trial-and-error.
  • Docker Desktop 4.34 host networking: https://www.docker.com/blog/docker-desktop-4-34/

Proposed Solution

Proposed Solution

ZecDev Launchpad provides a maintained developer + CI layer that standardizes how Zcash builders bring up Zebra (regtest) with a faucet, Unified Address (ZIP-316) fixtures, and either a lightwalletd or Zaino backend; locally and in CI, then runs golden end-to-end shielded flows to catch breakage early.

How it closes the gaps :

- One-command devnet for the post-`zcashd` era. A single CLI command starts Zebra (regtest) + faucet + UA fixtures + your chosen backend, replacing fragile, per-repo scripts with a consistent workflow aligned to the `zcashd` → Zebra transition.
  Refs: Zcash deprecation; Zebra Docker quick start.
- DX/CI standardization (beyond "how to run lwd" docs). Zebra documents running lightwalletd and ships a `docker-compose.lwd.yml`, but that's not a full developer workflow. Launchpad adds pre-mined funds, health checks, UA fixtures, golden flows, and CI artifacts; the missing pieces teams keep rebuilding.
- Backend parity while the stack evolves. Many projects still use lightwalletd; Zaino is an emerging Rust indexer. Launchpad exposes a simple toggle (`--backend=lwd|zaino`) and runs the same tests on both, so migrations don't lose coverage.
- UA-centric E2E tests that mirror real usage. Built-in flows exercise generate UA → fund → autoshield → shielded send → rescan/sync → verify, making ZIP-316 behavior visible in CI and catching interoperability bugs earlier.
- Security-first defaults and explicit guardrails. Services bind to `localhost`; any testing-only flags (e.g., the "no-TLS for testing" pattern in upstream compose) are opt-in with loud warnings.
- Linux-first reliability; Desktop caveats documented. CI predominantly runs on Linux. We treat Linux as GA and document Docker Desktop host networking (4.34+) as opt-in on macOS/Windows with clear workarounds.

What we will ship:

- `zecdev` CLI: `up` (Zebra+faucet+backend, health checks, UA fixtures, pre-mined funds), `test` (golden flows), `down` (clean teardown).
- Reusable GitHub Action: Spins the same stack per PR, runs UA flows, publishes logs/artifacts, supports `backend: lwd|zaino`.

- Reference E2E suite: UA vectors, sync/rescan edge cases, backend-parity checks.
- Docs & quickstarts: “2-minute local start,” “5-line CI,” compatibility matrix (Zebra/NU + backends), and Desktop caveats.

Why this solves it

Instead of every team hand-rolling brittle scripts, Launchpad productizes the exact workflow builders need now is a Zebra + a light-client backend + UA-centric E2E in CI so regressions surface before merge, UA behavior stays correct, and backend migrations don’t sacrifice test coverage.

Solution Format

Type: Open-source software (tooling + CI) with documentation and a sample repository.

Core Deliverables

- CLI Tool - `zecdev` (Linux-first)
  - `zecdev up --backend=lwd|zaino` → starts Zebra (regtest) + faucet + selected backend with health checks, version pinning, pre-mined test funds, and Unified Address (ZIP-316) fixtures.
  - `zecdev test` → runs golden end-to-end flows (generate UA → fund → autoshield → shielded send → rescan/sync → verify).
  - `zecdev down` → clean teardown and log collection.
  - Safety defaults: services bind to `localhost`; any testing-only flags are opt-in with explicit warnings. No mainnet keys or funds.
- Reusable GitHub Action - `zecdev-devnet`
  - Action packaged for GitHub Marketplace publication as part of delivery.
  - Inputs: `backend: lwd|zaino`, timeouts, chain params. Outputs: structured logs/artifacts for CI triage.
  - Example workflow YAML included for drop-in use.
- Reference E2E Test Suite & Fixtures
  - UA vectors (ZIP-316), sync/rescan edge cases, and backend-parity checks (lightwalletd ↔ Zaino).
  - Reproducible funding/spend recipes; failure repro harness.
  - Note: Zaino support is delivered and marked experimental initially, pending upstream API stability.
- Documentation & Quickstarts

    - “2-minute local start,” “5-line CI,” troubleshooting, and a compatibility matrix (Zebra versions/NU heights, backends).
    - Clear guidance for Docker Desktop networking on macOS/Windows, with known limitations and workarounds.
    - Security notes (testing flags, local bindings, secrets handling).
- Sample Repository
    - Minimal public repo wired to the Action (both backends), demonstrating green CI runs.
    - Serves as a reference implementation teams can copy or adapt.

Release & Distribution

- Licensing: MIT / Apache-2.0 dual license.
- Releases: Versioned GitHub releases (CLI binaries for Linux x86_64/arm64). Pinned Action tags (e.g., `v1`, `v1.x`).
- Artifacts: CI logs and redacted network traces (where applicable) to support debugging without exposing sensitive data.

Maintenance Window

- 90-day maintenance after initial release for version bumps (Zebra/NU, backend updates), compatibility fixes, and docs updates.
- Monthly status notes. Handover plan to invite wallet/SDK maintainers to co-maintain post-window.

Dependencies

1) Technical software & runtime

- Operating system
    - Linux (primary): Ubuntu 22.04+ or equivalent.
    - macOS/Windows (best-effort): Docker Desktop 4.34+ (for optional host-networking). Desktop networking behavior can differ from Linux and may require configuration.
- Container stack
    - Docker Engine ≥ 24.x and Docker Compose v2.
    - Ability to pull public container images (e.g., GHCR/Docker Hub).
- CI environment
    - GitHub Actions (e.g., `ubuntu-latest`) or a self-hosted Linux runner.

    - Permissions to upload artifacts (logs/run summaries).
- Zcash components (version-pinned by this project)
    - Zebra (`zebrad`) in regtest mode.
    - One light-client backend selectable at runtime:
        - lightwalletd (documented integration with Zebra), or
        - Zaino (experimental status reflected in config and docs).
    - Unified Address (ZIP-316) test fixtures bundled with the project.
- Build toolchain (for contributors)
    - Recent Rust stable toolchain if building `zecdev` from source. (Prebuilt Linux binaries will be provided at release; source build remains available.)
- Local resources (typical)
    - ~2–5 GB disk for images/logs/fixtures.
    - ≥ 2 CPU cores / 4 GB RAM recommended for smoother runs.
- Networking
    - Local TCP ports for `zebrad` and the selected backend; bound to localhost by default.
    - Testing-only flags remain opt-in with explicit warnings; no public exposure is required.

2) Documentation & specs referenced

- Zebra + Docker quick start and examples: https://zebra.zfnd.org/user/docker.html
- Zebra + lightwalletd usage guidance: https://zebra.zfnd.org/user/lightwalletd.html
- ZIP-316 (Unified Addresses): https://zips.z.cash/zip-0316
- Docker Desktop host-networking (for Desktop best-effort): https://www.docker.com/blog/docker-desktop-4-34/

3) 4) Access & credentials

- GitHub account with permissions to enable Actions and upload artifacts.
- (Optional) Access tokens for private forks if testing in private repos.
- No mainnet keys or sensitive secrets are required; the toolkit uses regtest with pre-mined funds and binds to localhost by default.

Technical Approach

We will deliver a Linux-first, reproducible devnet + CI harness around Zebra, implemented as a small Rust CLI and a reusable GitHub Action. The stack runs `zebrad` in regtest with a faucet and toggleable light-client backends ; lightwalletd (documented pairing with Zebra) or Zaino (an emerging Rust indexer) and executes UA-centric end-to-end flows in CI.

1) Architecture

- Core node: `zebrad` in regtest for fast, isolated testing with configurable upgrade heights. :contentReference[oaicite:1]{index=1}
- Light-client backend (runtime toggle): `--backend=lwd|zaino`
  – lightwalletd backed by Zebra (officially documented). :contentReference[oaicite:2]{index=2}
  – Zaino (Rust indexer) shipped as experimental while APIs mature. :contentReference[oaicite:3]{index=3}
- Developer surface: `zecdev` CLI (Rust) + Docker Compose v2.
- CI surface: GitHub Action that brings up the same stack per PR and runs golden flows.

2) Orchestration & health

- Compose profiles to start Zebra + faucet + selected backend, with readiness probes (RPC/HTTP/gRPC).
- Version pinning and a public compatibility matrix (Zebra version/NU, backend versions).
- Seeded state: pre-mined spendable funds and UA fixtures for deterministic tests.
- We build on Zebra's own `docker-compose.lwd.yml` example but add health checks, fixtures, and CI artifacts to standardize DX beyond the example.

3) Golden end-to-end methodology

- Flows mirror real usage: generate UA → fund → autoshield → shielded send → rescan/sync → verify.
- Tests run locally via `zecdev test` and in CI via the Action; failures upload structured logs for triage.
- UA behavior is validated against ZIP-316 expectations and ecosystem guidance.

4) Security posture (testing focus)

- Local-only by default: bind services to `localhost`; no public exposure required.
- Testing-only flags (e.g., the no-TLS setting in Zebra's lwd compose) remain opt-in with explicit warnings in the CLI/docs.

5) Platform support

- Linux = GA (matches most CI runners).
- macOS/Windows = best-effort with documented Docker Desktop host-networking (supported in Desktop 4.34+) and known caveats; Linux remains the reference path.

6) Maintenance & versioning

- Pinned container tags for Zebra/backends; automated bump PRs during the maintenance window.
- Matrix CI (optional): run both backends to catch parity regressions early as the ecosystem continues migrating from `zcashd` to Zebra.

Upstream Merge Opportunities

This project is integration-focused. We expect upstream contributions to be docs/examples and small DX hooks rather than protocol changes.

Target upstream repositories

- ZcashFoundation/zebra — node + docs ("The Zebra Book"): https://zebra.zfnd.org/
- zcash/lightwalletd — light-client backend: GitHub - zcash/lightwalletd: Lightwalletd is a backend service that provides a bandwidth-efficient interface to the Zcash blockchain
- zingolabs/zaino — Rust indexer (used here as an experimental backend): GitHub - zingolabs/zaino: Zaino is an indexer for the Zcash blockchain implemented in Rust.

Planned changes

- Zebra (docs/examples)
  - Add a DX-oriented compose profile for local devnet (Zebra regtest + faucet + backend placeholder), with health checks, localhost bindings, and clear notes around any testing-only flags.
    ↳ Context: Zebra's Docker & lwd docs and example compose:
    Zebra with Docker - The Zebra Book • Lightwalletd - The Zebra Book • https://raw.githubusercontent.com/ZcashFoundation/zebra/main/docker/docker-compose.lwd.yml
  - Expand regtest quick-start snippets: pre-mined funds, UA (ZIP-316) fixtures, and a short "golden flow" example (generate UA → fund → shielded send → rescan/sync → verify).
- lightwalletd (docs + small readiness guidance)
  - Document reliable readiness/health checks (ports/endpoints) for CI, reference safer local defaults, and link a minimal compose for Zebra-backed runs.
    Repo: GitHub - zcash/lightwalletd: Lightwalletd is a backend service that provides a bandwidth-efficient interface to the Zcash blockchain
- Zaino (docs/examples)
  - Provide a minimal compose and health-check note to run Zaino alongside Zebra in CI; label examples experimental to reflect ongoing API evolution.
    Docs/Repo: Zaino Workspace - Rust • GitHub - zingolabs/zaino: Zaino is an indexer for the Zcash blockchain implemented in Rust.

Ecosystem benefit

- Lower setup time for new contributors by providing copy-paste examples that match common dev workflows (not just ops setups).
- Reduce CI flakiness via consistent health checks and localhost defaults.
- Enable backend parity testing (lightwalletd ↔ Zaino) without bespoke scripts.

Coordination

- Zebra maintainers (ZF): placement and wording for new examples, security callouts around testing flags.
- lightwalletd maintainers (zcash org): confirm recommended readiness checks/ports used in CI.
- Zingolabs (Zaino): confirm a minimal health probe and note experimental status.

Timeline (indicative, tied to our milestones)

- After our CLI/compose MVP (M2): PRs to Zebra (devnet compose + regtest quick-starts) and lightwalletd (readiness/docs).
- After CI/E2E suite (M3): PRs to Zaino (compose + health docs) and a follow-up Zebra docs link to the public GitHub Action usage.
- Maintenance window: small doc fixes and version-pin updates based on early adopter feedback.

Service Costs Justification

To fund a small Linux VM (self-hosted Actions runner) that ensures stable, reproducible multi-container devnet tests (Zebra + faucet + lwd/Zaino), enables backend-parity matrix runs, and avoids shared-runner variance/queue delays during development and the 90-day maintenance window.

Milestone Details

- Milestone: 1
  User Stories:
  - “As a wallet/SDK developer, I want a one-command local Zebra devnet so I can test shielded flows without custom scripts.”
  - “As a CI maintainer, I want a minimal, reliable smoke test so PRs fail fast when the devnet is unhealthy.”
    Deliverables:
  - Technical spec and acceptance tests documented in repo; repository bootstrapped (license, CONTRIBUTING, issue templates).
  - Initial compose profiles (Zebra regtest + faucet + backend placeholder), health-check plan, localhost defaults.
  - Self-hosted Linux CI runner provisioned; baseline CI smoke job.
    Acceptance Criteria: On a fresh Linux VM, the scaffolded zecdev up brings up containers, health checks pass, and acceptance-test issues are published.
- Milestone: 2
  User Stories:
  - “As a developer, I want zecdev up/test/down so I can run end-to-end shielded flows locally.”

    - “As an integrator, I want to choose lightwalletd or Zaino so I can test against my backend.”
      Deliverables:
    - zecdev CLI (Linux-first): up starts Zebra (regtest) + faucet + backend with health checks, version pins, pre-mined funds, UA (ZIP-316) fixtures; test; down.
    - Backend toggle: --backend=lwd|zaino (Zaino marked experimental); Desktop caveats documented.
      Acceptance Criteria: On a fresh Linux VM, zecdev up --backend=lwd and zecdev up --backend=zaino both reach healthy; zecdev test completes a basic shielded send; logs are captured.
- Milestone: 3
  User Stories:
    - “As a repo maintainer, I want a reusable GitHub Action so PRs run the same golden flows automatically.”
    - “As an engineer, I want artifacts on failure so I can diagnose issues quickly.”
      Deliverables:
    - GitHub Action published (inputs: backend/timeouts/chain params; outputs: logs/artifacts).
    - Golden E2E flows: generate UA → fund → autoshield → shielded send → rescan/sync → verify.
    - Sample public repo wired to the Action with a backend matrix (lwd + Zaino).
      Acceptance Criteria: Action is visible on the Marketplace; sample repo shows a passing CI run on lightwalletd and an executing (experimental) Zaino job; an injected failure produces artifacts.
- Milestone: 4
  User Stories:
    - “As a new contributor, I want a 2-minute local start so I can get my first passing test quickly.”
    - “As a maintainer, I want a 5-line CI setup so I can enable checks with minimal changes.”
      Deliverables:
    - Docs: quickstarts, troubleshooting, compatibility matrix (Zebra/NU + backends), security notes (localhost, test flags opt-in).
    - Short demo video; forum post inviting feedback.
      Acceptance Criteria: From a clean VM, following docs yields a passing local run in 10 minutes or less and a passing CI run on the sample repo; links are public.

- Milestone: 5
  User Stories:
  - “As an adopter, I want timely version bumps so my CI stays green as Zebra/backends update.”
  - “As a contributor, I want responsive triage so issues get unblocked.”
    Deliverables:
  - 90-day maintenance: version pin updates (Zebra/backends), small fixes, monthly status notes.
    Acceptance Criteria: Matrix CI remains green on current Zebra/backends; at least two monthly status notes posted; in-scop